

Add-ons & Packaging

Kubernetes Production · Lesson 19

Cluster add-ons via Kubespray + Helm & Kustomize

kubespray.io · helm.sh · kubernetes.io/docs

Learning Objectives

By the end of this lesson you will be able to:

- Separate **platform add-ons** (Kubespray) from **application packaging** (you)
- Toggle cluster add-ons in `addons.yml` and reconcile with `cluster.yml`
- Configure **cert-manager**, the **registry** and **MetalLB** (L2 / BGP)
- Explain why **ingress-nginx / dashboard** were removed → install via **Helm**
- Package apps with **Helm** (Chart / Repo / Release / Values, no Tiller)
- Customise manifests with **Kustomize** (bases, overlays, generators, patches)

Reference: *Kubespray add-ons · Using Helm · Kustomization* —
kubespray.io · helm.sh · kubernetes.io/docs

19.1 Two Layers: Add-ons vs Packaging

Two Layers of "Extra Stuff" on a Cluster

There are **two distinct layers** — at **different times, different tools**:

Layer	What	Who / When	Tool
Platform add-ons	metrics-server, MetalLB, cert-manager, registry...	Kubespray, at deploy time	<code>addons.yml</code> → <code>cluster.yml</code>
Application workloads	<i>your</i> apps + third-party components	You, at run time	Helm / Kustomize

- **Kubespray** `addons.yml` installs *platform* add-ons **as part of the cluster** — re-run `cluster.yml` and Ansible converges the desired state.
- **Helm / Kustomize** are how *you* deploy and customise *application* workloads on the **running** cluster's API server.

Resolving the overlap: `helm_enabled: true` does **not** deploy your apps — it only makes the **Helm tooling available**. Two layers, two times, no overlap.

19.2 Cluster Add-ons via Kubespray

The `addons.yml` File

Add-ons are **optional cluster services** Kubespray deploys as part of `cluster.yml`. Each is an **on/off boolean** in one inventory file:

```
inventory/sample/group_vars/k8s_cluster/addons.yml
```

```
# 1. edit the toggle → 2. re-run the playbook → 3. add-on appears
ansible-playbook -i inventory/mycluster/hosts.yaml cluster.yml
# or scope it: --tags metallb
```

- **Provision-at-deploy-time:** you do **not** `helm install` these — `cluster.yml` reconciles them.
- **Declarative & idempotent** — same model as the rest of Kubespray: the file describes desired state, the playbook converges to it.

Common Toggles

All booleans, in `group_vars/k8s_cluster/addons.yml`:

Toggle	Add-on
<code>helm_enabled</code>	Helm client on control-plane nodes (charts post-deploy)
<code>metrics_server_enabled</code>	metrics-server — <code>kubectl top</code> + HPA
<code>cert_manager_enabled</code>	cert-manager — certificate controller
<code>ingress_nginx_enabled</code>	ingress-nginx — removed from recent releases
<code>metallb_enabled</code>	MetalLB — <code>type: LoadBalancer</code> on bare metal
<code>local_path_provisioner_enabled</code>	dynamic local-disk PVs (Rancher)
<code>registry_enabled</code>	in-cluster private Docker registry
<code>dashboard_enabled</code>	Kubernetes Dashboard — also removed

```
# addons.yml
metrics_server_enabled: true
helm_enabled: true
cert_manager_enabled: true
```

cert-manager (`cert_manager_enabled`)

A **native Kubernetes certificate controller** — issues from **Let's Encrypt**, **Vault**, **Venafi**, **signing key pairs**, or **self-signed**, and **renews before expiry**.

- **Secures Ingress** via the **ingress-shim**: annotate the Ingress and the shim **auto-creates the Certificate**; certs **rotate ~every 3 months**, stored as **Secrets**.

```
metadata:
  annotations:
    cert-manager.io/cluster-issuer: ca-issuer
```

- **Issuers**: an **ACME Issuer** = an account at an ACME CA (e.g. Let's Encrypt); ACME certs are **free + trusted by default**, validated via **HTTP01 / DNS01**.
- **Trust an internal CA**: set `cert_manager_trusted_internal_ca` (a PEM block) in `addons.yml`; wire your root CA (built with **cfssl**) as a **ClusterIssuer**.

In-Cluster Registry (`registry_enabled`)

A **private Docker registry as a Pod** — store proprietary images with **no external dependency**. Vars: `registry_namespace`, `registry_storage_class`, `registry_disk_size`. Runs in `kube-system`.

- **Storage: persistent** via a PV/PVC, or **temporary** via `emptyDir`.
- **Exposure:** a **Service on port 5000** + a **DaemonSet proxy on every node** binding `hostPort: 5000` → each node reaches it at `localhost`.
- **Why the proxy:** the registry has **no SSL or auth** (Docker would reject it as **"insecure"**). Running the proxy on `localhost` makes Docker treat it as **secure**.

```
# reference images via the localhost proxy
docker push localhost:5000/user/container
```

MetalLB — Bare-Metal LoadBalancer (`metallb_enabled`)

Lets you create `type: LoadBalancer` Services on bare metal (no cloud).

```
# addons.yml
metallb_enabled: true
metallb_speaker_enabled: true
metallb_namespace: metallb-system
kube_proxy_strict_arp: true          # PREREQUISITE
```

- **Strict ARP** stops MetalLB answering unwanted ARP queries from the `kube-ipvs0` interface — **required**.
- **Address pools** (`metallb_config.address_pools`): `ip_range` as **CIDR** (`192.0.3.0/24`) or **dash form** (`192.0.1.0-192.0.1.254`), `auto_assign` (default `true`), `avoid_buggy_ips: true` (skip `.0` / `.255`).

Calico ≥ 3.18: set `metallb_speaker_enabled: false` and use `calico_advertise_service_loadbalancer_ips` (Calico does the BGP advertising).

MetalLB — L2 vs BGP (address pools)

```
metallb_config:
  address_pools:
    primary:
      ip_range:
        - 192.0.3.0/24
      auto_assign: true
      avoid_buggy_ips: true
  layer2:                                # L2 (default): one node answers via ARP
    - primary
  layer3:                                # BGP / Layer3: announce to routers
    metallb_peers:
      peer1:
        peer_address: 192.0.2.1
        peer_asn: 64512
        my_asn: 64513
        # optional: peer_port (179), hold_time (120s),
        # password (TCP-MD5), router_id, communities, address_pool
```

- **Layer2** — simple ARP, one node owns the VIP. **BGP** — advertise service IPs to upstream routers via `metallb_peers`.

Local Storage + Removed Add-ons → Helm

Local storage toggles:

- `local_path_provisioner_enabled` — Rancher provisioner, **dynamic** local-disk PVs from a node path (simple default for labs).
- `local_volume_provisioner_enabled` — **static** (`kubernetes.io/no-provisioner`): one PV per mount under `host_dir`; **you pre-create the mounts** and persist them via `/etc/fstab`.

Removed add-ons → install with Helm:

- In **recent Kubespray versions**, `ingress_nginx_enabled` and `dashboard_enabled` **were removed** from the built-in set.
- Recommended replacement: **install them with Helm after deploy** — which is exactly why `helm_enabled: true` exists.

19.3 Helm — Packaging Your Workloads

Helm — Three Concepts + Helm 3

Helm is the **package manager for Kubernetes** — bundle, version, template, upgrade and roll back sets of resources.

Concept	Meaning
Chart	A Helm package — all resources to run an app (like apt/yum)
Repository	Where charts are shared (indexed HTTP server or OCI registry)
Release	A running instance of a chart — one chart → many releases

- **Values** parameterize a chart's templates.
- **Helm 3 is client-only — no Tiller.** The client renders templates **locally** and submits manifests **directly to the API server** using your **kubeconfig + RBAC**. Release state is stored as **Secrets** in the release's namespace.

Chart Structure & Chart.yaml

```
mychart/  
├── Chart.yaml      # REQUIRED – chart metadata  
├── values.yaml     # default configuration values  
├── charts/         # subcharts (dependencies)  
├── crds/           # CRDs installed before the rest of the chart  
└── templates/      # Go-templated Kubernetes manifests
```

```
# Chart.yaml  
apiVersion: v2      # v2 = Helm 3  
name: mychart  
version: 1.2.3      # CHART version – SemVer 2 (required)  
appVersion: "1.16.0" # version of the packaged APP (informational)  
type: application
```

- **version** = the chart's own version; **appVersion** = the app's version — **independent**. Defaults live in `values.yaml`; manifests in `templates/`.

Helm Workflow

```
# Repositories
helm repo add bitnami https://charts.bitnami.com/bitnami
helm repo update           # refresh cached index
helm search repo nginx     # search added repos

# Install / upgrade / uninstall
helm install my-release bitnami/nginx
helm upgrade --install my-release bitnami/nginx # idempotent deploy
helm uninstall my-release

# Inspect & roll back
helm list -A               # releases in all namespaces
helm history my-release    # revision history
helm rollback my-release 1 # revert to revision 1
helm template my-release bitnami/nginx # render, do NOT install
```

helm upgrade --install is the idempotent CI/CD deploy verb;
helm template renders manifests **without** touching the cluster.

Overriding Values & Precedence

```
# YAML file(s) of overrides — later -f wins
helm install my-release bitnami/nginx -f base.yaml -f prod.yaml

# Inline overrides — rightmost --set wins
helm upgrade --install my-release bitnami/nginx --set image.tag=1.25
```

Value precedence (lowest → highest):

1. chart's default `values.yaml`
2. parent chart values (for subcharts)
3. user `-f` / `--values` files (later file overrides earlier)
4. `--set` flags — **highest priority, override everything**

CKA: `--set > -f > values.yaml`. Useful flags: `-n` / `--namespace`, `--create-namespace`, `--dry-run`, `--debug`, `--version`.

19.4 Kustomize — Customising Manifests

Kustomize — Template-Free, in kubectl

- **Template-free, declarative** customization of raw Kubernetes YAML — no Go templating, no DSL. You keep **valid manifests** and layer changes on top.
- **Built into `kubectl` since v1.14** — `kubectl apply -k` and `kubectl kustomize` need **no extra binary**.
- Driven by one `kustomization.yaml` listing **resources** plus **generators** (create objects) and **transformers** (mutate objects).

Field	Category	Purpose
<code>resources</code>	compose	Manifest files / dirs / bases to include
<code>configMapGenerator</code> / <code>secretGenerator</code>	generate	Build from literals / files
<code>namespace</code>	transform	Set namespace on all resources
<code>namePrefix</code> / <code>nameSuffix</code>	transform	Prefix/suffix all names
<code>labels</code>	transform	Add labels (and selectors) — replaces <code>commonLabels</code>
<code>images</code>	transform	Override <code>newName</code> / <code>newTag</code> / digest
<code>patches</code>	patch	Modify specific fields of specific resources

Generators & Transformers

```
# kustomization.yaml
resources:
  - deployment.yaml
  - service.yaml

namespace: my-namespace
namePrefix: dev-
labels:                                     # use labels: (NOT deprecated commonLabels)
  - pairs:
      app: bingo

configMapGenerator:
  - name: app-config
    literals:
      - LOG_LEVEL=debug

images:
  - name: nginx
    newTag: 1.25-alpine
```

Generators append a **content hash** to the name by default (a config change rolls the Pods); disable with `generatorOptions.disableNameSuffixHash: true`.

Bases & Overlays

A **base** holds shared manifests; an **overlay** references it and patches it per environment — keeping the base DRY.

```
# overlays/prod/kustomization.yaml
resources:
  - ../../base          # reference the base
namespace: production
namePrefix: prod-
patches:
  - path: replica-patch.yaml    # strategic-merge patch
images:
  - name: myapp
    newTag: v1.2.0
```

```
kubectl kustomize overlays/prod    # render to stdout (preview, no apply)
kubectl apply -k overlays/prod     # build AND apply
kubectl delete -k overlays/prod    # delete the built resources
```

19.5 Helm vs Kustomize

Helm vs Kustomize — When to Use Which

Aspect	Helm	Kustomize
Approach	Package manager + templating	Template-free overlays/patches
Unit	Versioned chart , tracked release	Plain YAML + <code>kustomization.yaml</code>
Tooling	Separate <code>helm</code> binary	Built into <code>kubectl</code> (<code>-k</code>)
Customization	<code>values.yaml</code> , <code>--set</code> , <code>-f</code>	bases + overlays, generators, patches
Lifecycle	install / upgrade / rollback / history	<code>apply -k</code> / <code>delete -k</code> (no rollback)
Best for	Third-party cluster components & complex apps	Tweaking your own manifests per env

- Use **Helm** for off-the-shelf components (ingress controllers, monitoring) and apps that benefit from **templating + rollbacks**.
- Use **Kustomize** to maintain **your own** manifests across dev/staging/prod.
- They **compose** — post-render a Helm chart with Kustomize, or install dependencies with Helm alongside Kustomize-managed manifests.

Resolving the Overlap

The two layers map onto **time** and **ownership**:

- **addons.yml (Kubespray)** → *platform* add-ons, installed **as part of the cluster**, at **deploy time**, via **cluster.yml** (Ansible converges state).
- **Helm / Kustomize (you)** → *application* workloads, deployed at **run time** against the **running** API server.
- **helm_enabled: true** does **not** deploy your apps — it just makes the **Helm tooling available** on the control plane.

The removed add-ons (**ingress-nginx**, **dashboard**) are the bridge: they *used* to be platform toggles, now you install them as **application** charts with Helm — same cluster, different layer.

Key Takeaways

- **Two layers:** Kubespray `addons.yml` = *platform* add-ons; Helm/Kustomize = *your* apps
- Add-ons are **booleans in** `addons.yml`, reconciled by `cluster.yml` (idempotent)
- **MetalLB** = `type: LoadBalancer` on bare metal — **L2 or BGP**; set `kube_proxy_strict_arp: true`
- **ingress-nginx / dashboard removed** → install via **Helm**; that's what `helm_enabled` is for
- **Helm 3 = no Tiller**, client-only; state in **Secrets**; `upgrade --install` is idempotent
- **Value precedence:** `--set` > `-f` > `values.yml`
- **Kustomize** = `kubectl apply -k`, base + overlays, `labels:` (not `commonLabels`)

End of Lesson 19

Next: Lesson 20 — Logging, Monitoring & Troubleshooting

`addons.yml` · `ansible-playbook cluster.yml` · `helm upgrade --install` · `helm rollback` · `kubectl apply -k`